Building a CNN for Rock-Paper-Scissors Classification

Luca Rigamonti 08920A

28th february 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

This project applies classification model known as Convolutional Neural Network (CNN) to distinguishing between images of Rock, Paper, and Scissors gestures. Several different CNN architectures were tested to determine which one performed the best on the input data.
Once the best model was identified, a risk estimate was applied through cross-validation to validate the performance of the model.

2 Data

The project is based on the Rock Paper Scissors dataset, a popular resource for teaching neural networks how to distinguish between different hand gestures.
The dataset contains a total of 2188 images corresponding to the 'Rock' (726 images), 'Paper' (710 images) and 'Scissors' (752 images) hand gestures of the Rock-Paper-Scissors game. All image are taken on a green background with relatively consistent ligithing and white balance.
All images are RGB images of 300 pixels wide by 200 pixels high in .png format. The images are separated in three sub-folders named 'rock', 'paper' and 'scissors' according to their respective class.
To begin with, the data were unzipped, and the input data were used to create a pandas DataFrame in which each image was labeled according to whether it depicted a rock, paper or scissor's image. Creating a DataFrame allows me to efficiently keep track of the labels and corresponding file names for each image in the dataset.

	paths	label
1037	/content/rps-cv-images/rock/zdNuxse3BvJK3s6Q.png	rock
243	/content/rps-cv-images/paper/hH7jcOkAiHqNDWcy.png	paper
583	/content/rps-cv-images/paper/aOiWbMZF8BhxwLTs.png	paper
1777	/content/rps-cv-images/scissors/hJuytZrZ7fR1CW...	scissors
610	/content/rps-cv-images/paper/8cUuoeM7qSsAGoOv.png	paper

Figure 1: A sample block of the Pandas DataFrame showing the mapping between image file paths and their corresponding labels (rock, paper, scissors)

Moreover, it makes also easier to split the dataest into training, validation, and testing sets.

2.1 Image resizing and Normalization

For the preprocessing phase, two key transformations were applied to the 'Rock Paper Scissors' dataset: image resizing and normalization. First, all images were downsampled to a resolution of 96x96 pixels to reduce computational overhead and ensure a consistent input shape for the convolutional layers.
Then, the pixel values were normalized using a Min-Max scaling approach (1/255), shifting the intensity range from $[0, 255]$ to $[0, 1]$. This normalization step is essential to prevent gradient instability and to accelerate the convergence of the model during training.

2.2 Data Splitting Strategy

The dataset was partitioned into three distinct subsets: **Training**, **Validation**, and **Testing**. The splitting process was done in two stages to get a balanced distribution:

- **Initial Split**: 10% of the total data was reserved as Test Set. This subset is used exclusively for the final evaluation to measure the model's performance.
- **Validation Split**: Out of the remaining 90% of the data, 11.1% was assigned to the Validation Set. This specific percentage ($0.111 \times 0.90 \approx 0.10$) was chosen to achieve a final split of **80% Training**, **10% Validation**, and **10% Test**.

3 Models

In this project I will evaluate three different models of convolutional neural network (CNN) in order to find which one has the best performance for this specific dataset and problem.
Specifically, I will test three distinct CNN architectures to identify the most effective model for this task. These versions explore various combinations of layers and filters while maintaining a similar underlying structure.

3.1 Hyperparameter tuning

To enhance the model's efficiency, I performed Hyperparameter Tuning. This step was essential to identify the ideal balance between model complexity and learning speed.
In this phase, I worked on the first model to identify the optimal configuration of its core hyperparameters. Specifically, I tuned the following parameters:

- **Learning Rate:** The step size used during the gradient descent to update the weights.

- **Number of Epochs:** The total number of times the entire training dataset is passed through the network.
- **Batch Size:** The number of training samples processed before the internal model parameters are updated.

The goal was to minimize the loss, but I evaluated the models using accuracy to better understand their real-world effectiveness..

The model optimization was carried out in three incremental stages:

1. **Trial 1 (Baseline):** Used default Adam learning rate ($\alpha = 0.001$) and 40 epochs. The accuracy plots showed high volatility after epoch 30, suggesting that the model was becoming unstable.
2. **Trial 2 (Reduced LR):** The number of epochs was capped at 30 and a custom learning rate of $\alpha = 0.0003$ was introduced. While performance improved, the validation curves still exhibited significant noise.
3. **Trial 3 (Final Model):** To achieve smoother convergence, the learning rate was further decreased (0.0001), and the *Batch Size* was fixed at 64. This configuration resulted in the most stable model, effectively minimizing overfitting and reaching a final validation accuracy of approximately 97%.

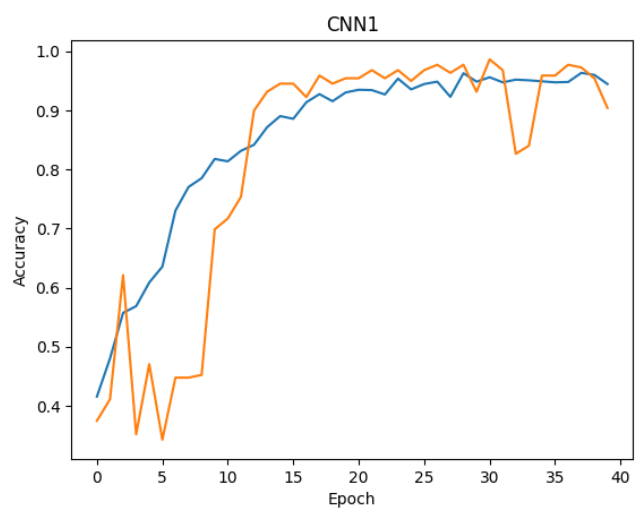

Figure 2: Trial 1

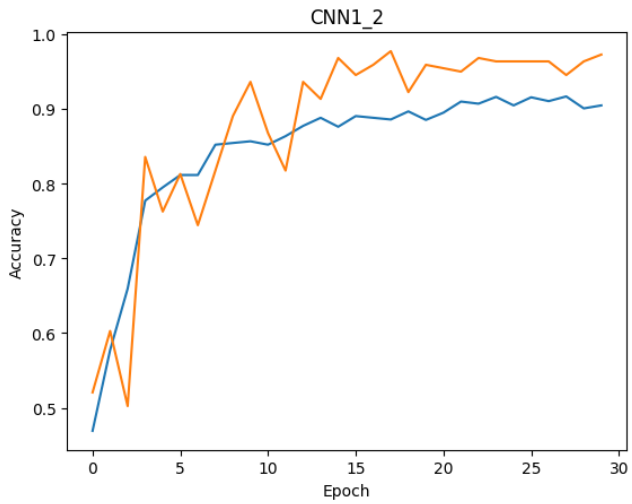

Figure 3: Trial 2

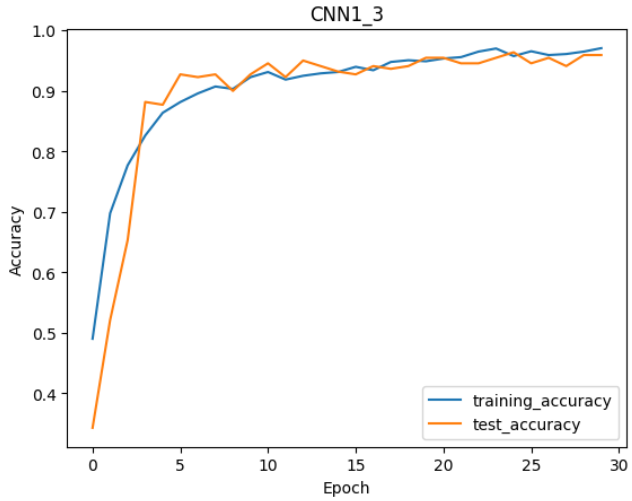

Figure 4: Trial 3

The hyperparameter tuning process was specifically conducted on the first model (**CNN1_3**). After identifying the optimal configuration that minimized volatility and maximized accuracy, the same set of parameters was applied to all models in this project:

- **Learning Rate:** $\alpha = 0.0001$
- **Batch Size:** 64.

- **Epochs:** 30.

3.2 Fixed Architectural Settings

While hyperparameters like learning rate and batch size were tuned, the following architectural features were kept **fixed** across all the models.

- **BatchNormalization:** Included by default in all models to ensure training stability from the first epoch.
- **MaxPooling2D (2, 2):** Used consistently as the standard downsampling strategy.
- **Dropout Strategy:** I fixed the dropout rates at 0.3 for the initial layers and 0.4 for the deeper layers to provide a constant level of regularization against overfitting.
- **Kernel Regularization (L2):** All models share a fixed penalty value of 0.005. Although I initially considered 0.01, I settled on 0.005 as a permanent setting to maintain a good balance between bias and variance without further modification.

In particular, the final dense layer, which provides the final prediction, was configured with three neurons and a softmax activation function (a standard choice for multiclass classification problems with more than two classesa as it returns a probability distribution).

3.3 Model 1

The first model (called CNN1.3 in the code) contains two convolutional layers (Conv2D), each composed of 64 filters.
As can be seen from the code, the CNN1 model is composed of two convolutional blocks using 64 filters each with $3x3$ kernels, supported by Batch Normalization.

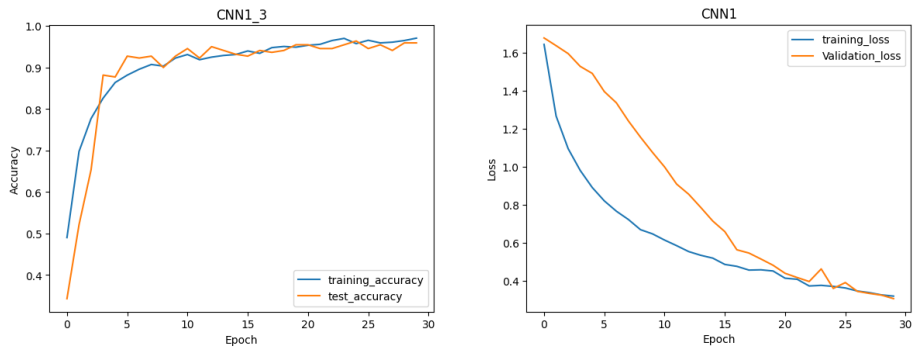

Figure 5: Model 1 Accuracy and Model 1 Loss

Metric	**Value**
Test Loss	0.3698
Test Accuracy	0.9680

3.4 Model 2

The second model (called CNN2 in the code) is composed of three convolutional blocks, each utilizing 32 filters with 3x3 kernels. As it can be seen from the code, the architecture increases the number of layers compared to the previous model while reducing the number of filters.

Metric	**Value**
Test Loss	0.6215
Test Accuracy	0.9635

The first model has a slightly better accuracy but the most significant difference is in Test Loss, where CNN1.3 (0.3698) is almost half that of CNN2 (0.6215). This indicates that CNN1.3 is much more confident and reliable in its predictions.

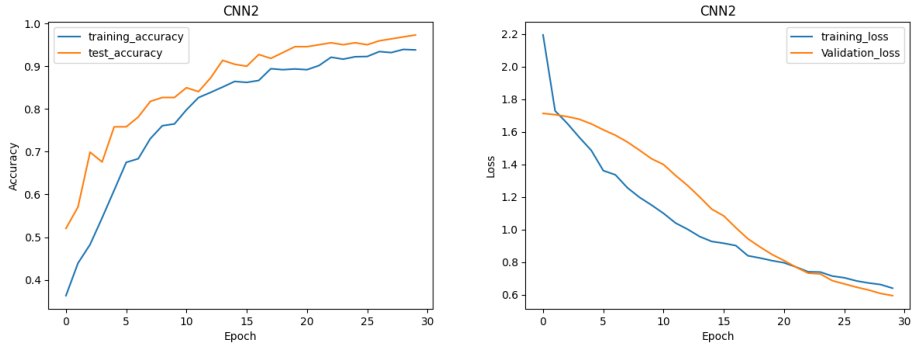

Figure 6: Model 2 Accuracy and Model 2 Loss

3.5 Model 3

The third model (CCn3 in the code) combines the depth of the previous architecture with the increased filter capacity of the first version. As it can be seen from the code, the model is composed of three different Convolutional Layers (CL), each utilizing 64 filters.

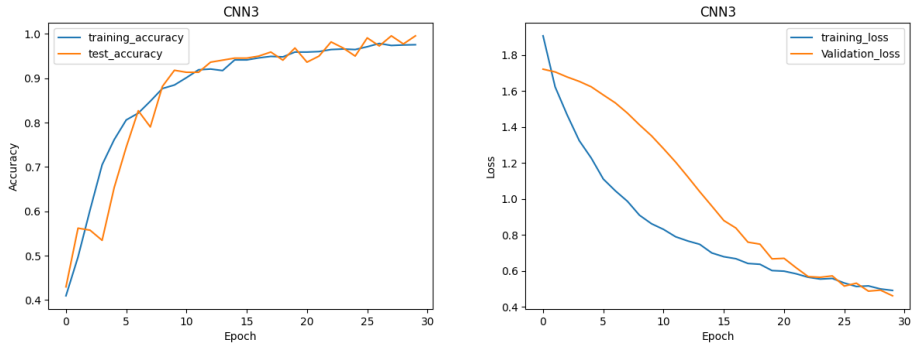

Figure 7: Model 3 Accuracy and Model 3 Loss

Metric	**Value**
Test Loss	0.5349
Test Accuracy	0.9680

4 Results

- The models with 64 filters (CNN1.3 and CNN3) achieve identical and higher accuracy (96.80) than the model with 32 filters (CNN2).
- CNN1.3 has a much lower Loss (0.3698 versus 0.5349). So, adding a third layer (in CNN3) increased the model's complexity without improving accuracy. On the contrary, it made the model less confident in its predictions (higher Loss), likely due to overfitting.
- Hyperparameter tuning played a key role in boosting the model's testing performance, as specifically demonstrated by the significant improvements observed in the first model.

So, CNN1.3 outperformed all other models with a peak accuracy of **96.80%** and the most stable loss curve. This makes it the most robust and reliable choice for the task of classifying images within the Rock-Paper-Scissors dataset, ensuring high precision in recognizing hand shapes.

4.1 Cross Validation

To verify that the previously obtained results (based on a single training and testing split) were indeed valid and not the result of a particularly favorable data split, a 3-fold cross-validation was implemented. This technique allows to test the model's robustness by evaluating its generalization ability across different portions of the dataset. For this analysis, a configuration with k = 3 was chosen, with training limited to 10 epochs for each fold. This choice was dictated by the need to balance statistical accuracuy with the high computational resources and computation times required to run all three models.

Model	**Average accuracy**	**Standard Deviation**
CNN_1_3	**0.9159**	**0.0029**
CNN_2	0.7907	0.0331
CNN_3	0.8743	0.0198

The data confirms that the first model is the best architecture. In addition to having the highest accuracy, it has the lowest standard deviation (0.0029), which indicates extreme stability.

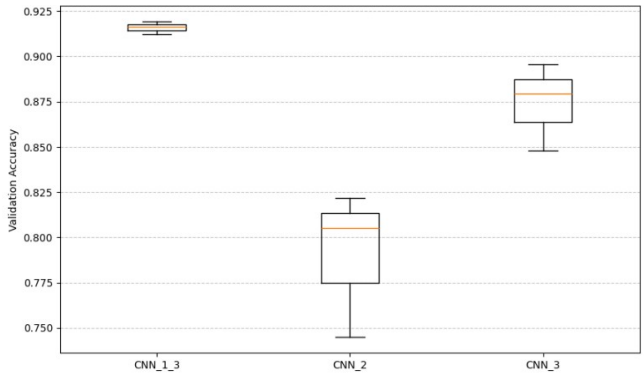

Figure 8: Box plot that illustrates the distribution of validation accuracy across the 3-fold cross-validation.

5 Appendix

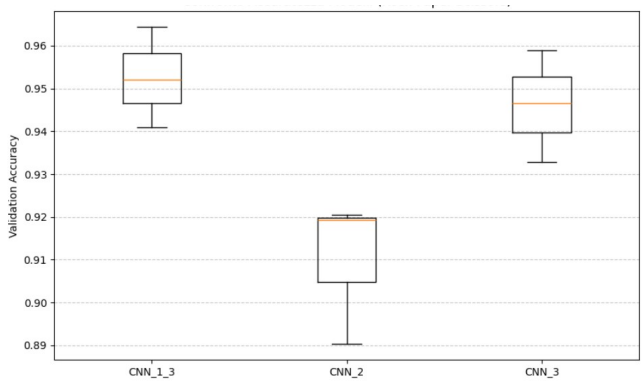

Figure 9: Box plot that illustrates the distribution of validation accuracy across the 3-fold cross-validation with 20 epochs. Not included in the project.